Find the Promos — a tool that reads a microdrama and picks its best ads

Kuku AI Generalist Assignment · Problem 1 | **Krish Agarwal** · Roll 228170543 · Earth Sciences (BS-MS), IIT Kanpur

Prototype: GitHub repo, run steps in README · *add link* • **Raw outputs:** `out/`, all 50 episodes · *add link* • **Walkthrough:** *add 3-min Loom link*

Why this problem, and who it serves

An editor hand-picks promo moments from every episode of every show, one at a time, and new shows ship weekly — the work grows linearly with the catalogue and is the clearest bottleneck of the three briefs. It also has the richest signal: timestamped screenplays I can read deterministically. Crucially, the audience is well-defined — US microdramas are driven by **women aged ~30–60, documented as especially drawn to romance and revenge narratives** (The Micro-Drama Economy report; TheWrap/Similarweb) — and this show is exactly that: a betrayal-at-the-altar revenge drama. That told me not just *where* to cut, but *which* moments this specific audience rewards. The tool proposes a *ranked shortlist of promo clips with exact in/out timestamps*, turning the editor's job from "watch everything and find" into "review a few and approve."

What it does

Point it at the scripts; it returns, per promo: episode, start/end timestamp of each segment, the **hook** (first 3s), the **cliffhanger** it cuts on, one line on why, and a 0–100 score — ranked across the show. On the pack it found **97 promos across all 50 episodes** in under a second; 28 carry a full wrong→payback arc.

How it works

Free, deterministic rules do the finding; judgment is isolated so a model can be swapped in:

- **Parse** — scripts are regular (type, speaker, emotion tag, in/out time per beat), so no AI is needed to read them.
- **Score each beat** — a cheap hook score (hot emotion, shock words, short punchy lines) and cliffhanger score (unanswered question, "cut to black", score swell, episode end).
- **Generate candidates** — for each strong hook beat, take the best cliffhanger 20–60s later; the window between is a candidate.
- **Judge** — craft rubric (hook 35 · cliffhanger 30 · stands-alone 20 · intensity 15), then an **audience-resonance layer** that lifts injustice→vindication clips (below). `--scorer llm` swaps the judge for an API model.
- **Dedupe & rank** — drop overlaps, keep best ~2/episode, sort all.

Why these numbers (grounded, not guessed)

- **3-second hook:** platforms treat the first 3s as a distinct ranking signal; ~40% of viewers swipe within it and >70% decide whether to stay (TikTok retention data).
- **20–60s length:** the platform sweet spot; ~72% of TikToks run under 60s and reach falls off past 90s.
- **Cliffhanger:** Loewenstein's *information-gap theory* (a knowledge gap is a cognitive "itch") and the *Zeigarnik effect* (unfinished tasks hold tension) — honest caveat: Zeigarnik's recall claim replicates weakly, so I lean on the tension, not memory.
- **Which emotions:** Berger & Milkman (2012, *JMR*) — high-arousal emotions travel, low-arousal sadness doesn't.
- **Audience-resonance:** Affective Disposition Theory (Weber et al. 2008, tested on 500+ *female* serial-drama viewers) — enjoyment peaks on poetic justice: a wronged, liked character vindicated / the villain punished.

Approach & scale

The expensive judgment (watching) is replaced by cheap rules that do the finding; a model is optional and only re-ranks a short list. The core has **zero dependencies** and runs instantly, so cost is dominated by the optional LLM call. At real volume — say 20 shows/week × 50 episodes, ~2 candidates each — that's ~2,000 short re-rank calls a week on a mini model: cents, not an editor-week. The model is a one-flag swap (`PROMO_LLM_MODEL` env var) and falls back to the free scorer if no key is set, so a batch never breaks and a stronger model you provide drops straight in. **The human stays in the loop:** the tool proposes ranked clips with exact timestamps; the editor approves or nudges the in/out points.

How I know it's good — and where it fails

I ran it on the full pack and read the top picks against the scripts. **#1 (Ep 38)** opens on "Do you even know what happened to me?" and carries a full wrong→payback arc; **#2 (Ep 18, 02:38–03:12)** opens on "Someone wanted us out." after the arson and cuts to black on a vow of vengeance. Adding the poetic-justice layer re-ranked 11 of the top 15 toward betrayal-and-vindication clips — the arcs this audience is documented to prefer — while keeping the verified-strong clips. Sanity: durations 20–59s (median 38), every episode covered, 36/97 cut on an unanswered question, 28/97 carry a full wrong+payback arc.

Where it picks badly	Why / fix
The resonance signal is keyword-based: it can *miss* poetic justice that is shown, not said. My first (aggressive) weight even buried the excellent Ep 18 clip — I dialled it back until it returned to #2.	This is the clearest case for the swappable **LLM scorer**, which reads meaning, not keywords. The weight itself is a hand-tuned judgment call.
"Stands-alone" is coarse (only returns 0.70 or 0.85).	Rules can't tell if a stranger follows the plot; an LLM judges this far better.
Purely visual hooks (a silent slap) score low; intensity is a weak differentiator.	Kept at low weight and flagged; audio/vision would fix it.

Tools used, and one place AI got it wrong

I used **Claude** to scaffold the parser/engine and pressure-test the scoring, and to gather the audience and psychology sources (which I then checked). One concrete miss: its first intensity metric scored almost every clip at ~0.10 — too harsh, and the scripts tag emotion mainly on dialogue — so intensity was a near-useless differentiator. Rather than tune it into a fake signal I dropped its weight and documented it. The scoring design, the 20–60s/continuous-clip decisions, grounding the weights in the audience + Affective Disposition Theory, and the hand-tuning of the resonance weight are mine.

Run: `python find_promos.py "<scripts_dir>" --out out` → `promos_ranked.md`, `promos.json`, `per_episode/`. Full citations in `PSYCH_NOTES.md`. The input pack is Kuku's and is not committed or hosted; the demo app takes uploads at runtime.